Static Type Checking of Concatenative Languages

Jaanus Pöial
Tallinn University of Technology

Abstract

Concatenative programs are assembled by juxtaposition, so their principal interface is not a tuple of named parameters but a transformation of an implicit stack. This paper gives a technical account of a source-level static checker for Forth-like concatenative languages. A program fragment is summarized by a pair of symbolic stacks. Symbols carry a type from an externally supplied finite subtype order and may carry indices that represent value correlations. Linear code is checked by boundary unification; alternative branches are combined by a partial lower-bound operation on compatible effects; and repeated code is approximated by comparing one and two executions. Parsing behavior, defining words, literals, and structured control forms are also externally specified.

The paper distinguishes the raw effect calculus from the implementation's compaction pass, gives executable definitions of the three core operators, and states an exact characterization of checking after a source body has been parsed. Compaction omits some implicit equality constraints, while the loop operator is a local one-versus-two stabilization test rather than a general fixpoint or Kleene closure. The account is based on the native `gforth-evaluator.fs` implementation and the checked-in corpus in this project. It also relates the design to the current WebAssembly 3.0 validation model and to recent work on first-class functions in statically typed concatenative languages.

Keywords: concatenative language; Forth; static analysis; stack effect; abstract interpretation; symbolic execution.

1 Introduction

In a concatenative language, juxtaposition is composition: if p and q are programs, then $p\,q$ executes p and passes its resulting stack directly to q. A conventional Forth stack annotation has the form

```
( n n -- n )
```

Such an annotation resembles a type signature, but an ordinary comment has no static force and does not generally express correlations between stack cells. For example,

```
DUP  ( x[1] -- x[1] x[1] )
OVER ( x[2] x[1] -- x[2] x[1] x[2] )
```

records facts that the unindexed signatures cannot express: two output positions may denote the same symbolic input.

This project turns such signatures into an executable abstract semantics. It loads three inputs: a finite type hierarchy, a vocabulary of word and parser specifications, and source text. It infers effects for sequences and newly defined words without executing the concrete program. The implementation is a single native Gforth source file, `gforth-evaluator.fs`.

The analysis addresses four problems in addition to stack-height consistency.

1. compatibility of an effect's output boundary with the next effect's input boundary;
2. propagation of subtype refinement and symbolic identity through stack shuffles;
3. construction of a common contract for alternative branches;
4. finite approximation of unbounded iteration.

The answers implemented here are, respectively, comparable-type unification, sequence-wide substitution, a partial effect meet, and a local repeated-effect operator. These ideas continue the algebraic development of stack effects in [5, 6, 7, 8, 9]. An earlier Java implementation of the framework was published in 2008 [10]; the implementation studied in this paper is the native Gforth version.

The analysis distinguishes five technical properties.

- The declared type relation need not be a complete lattice. The core algorithm requires only a finite normalized relation and selects a meet when the two types are comparable.
- Raw effects and printed effects are different domains. Normalization is deterministic, but it may erase an implicitly introduced two-occurrence correlation.
- Branch merging is an operation on contracts, not the set-theoretic union of branch transition relations.

- The implemented loop operation is $\pi \sqcap (\pi \cdot \pi)$. It is not a proof that the result describes every iteration count.
- Once parsing and leaf lookup are fixed, checking is the unique bottom-up evaluation of an acyclic system of effect equations.

2 Type and Symbol Domains

2.1 Normalized finite subtype relation

Let $\mathcal{T}$ be the finite set of canonical type names. The input permits aliases; mathematically, aliases are quotiented before considering the strict subtype declarations. Write $\tau \leq \upsilon$ when τ is at least as specific as υ. The bundled Forth-like profiles contain chains such as

$$a\text{-}addr < c\text{-}addr < addr < u < x, \qquad char < +n < n < x.$$

The implementation adds identity, inverse lookup information, and transitive relations after loading the declarations.

Definition 1 (Comparable type meet) *For $\tau, \upsilon \in \mathcal{T}$, define the partial operation*

$$\tau \sqcap \upsilon = \begin{cases} \tau, & \tau \leq \upsilon, \\ \upsilon, & \upsilon \leq \tau, \\ \textit{undefined}, & \textit{otherwise.} \end{cases}$$

Thus the implementation does not search for an arbitrary third subtype when τ and υ are incomparable. This is stricter than meet in a general lattice and identifies failures at the corresponding boundary.

2.2 Typed symbolic cells

A stack symbol is a triple

$$a = (\tau, i, e) \in \mathcal{T} \times \mathbb{N} \times \{0, 1\}.$$

It is printed as τ when $i = 0$ and as $\tau[i]$ otherwise. The bit e records whether an index was explicit in the specification. Symbols with the same positive index are correlated. An unindexed occurrence is freshened before composition, so two occurrences of plain `x` do not accidentally become equal.

This system uses indices as first-order equality variables, not as names of runtime storage locations. A substitution $[a \mapsto b]$ replaces every occurrence

of a in the current effect sequence. Global replacement is what allows a type fact discovered at one composition boundary to propagate through `DUP`, `SWAP`, or `OVER`.

3 Stack Effects and Frames

Let Σ be the set of typed symbols and let Σ^* be finite symbol sequences. The top of stack is written on the right.

Definition 2 (Raw effect) *A raw stack effect is a pair*

$$\pi = (L\ --\ R), \qquad L, R \in \Sigma^*.$$

It denotes a transformation of a visible stack suffix: for an arbitrary unchanged frame F, a stack of shape FL is transformed to FR, subject to the type and equality constraints represented by the symbols.

The empty effect $\mathbf{1} = (\ --\ )$ is identity. Algorithmic failure is represented by a separate clash value $\mathbf{0}$, not by an ordinary effect.

The implicit frame is essential. For example, $(n\ --n)$ does not assert that the whole runtime stack has exactly one cell; it constrains only the suffix touched by the word. Effects with different visible depths may consequently still be combined if their depth difference is a preserved prefix.

For raw effects, alpha-renaming positive indices has no meaning. Let $\pi \equiv_\alpha \pi'$ mean equality up to a bijective renaming of indices that preserves types and explicitness. Implementations must freshen independently instantiated word specifications before attempting unification.

4 Sequential Composition

Let

$$\pi_1 = (L_1 --R_1), \qquad \pi_2 = (L_2 --R_2).$$

Composition inspects the touching boundary from its top inward. If the current top symbols are $a = \tau[i]$ and $b = \upsilon[j]$, calculate $\rho = \tau \sqcap \upsilon$. If undefined, composition clashes. Otherwise make a fresh symbol $z = \rho[k]$, substitute a and b by z everywhere in both working effects, remove the matched output/input pair, and continue.

A direct functional presentation follows. All indices in the two operands are assumed to be disjoint. Let $\mathcal{C}$ denote raw composition:

$$\mathcal{C}(L_1, R_1, L_2, R_2) = \begin{cases} (L_2 L_1 - - R_2), & R_1 = \epsilon, \\ (L_1 - - R_1 R_2), & L_2 = \epsilon, \\ \mathcal{C}(L_1', R_1', L_2', R_2'), & \substack{R_1 = R_1' a, \\ L_2 = L_2' b}. \end{cases}$$

where the final case first applies the global substitution $a, b \mapsto z$ to all four sequences and then removes the displayed boundary pair. Concatenation order in the first case reflects the implicit frame: inputs still required by π_2 lie beneath those already required by π_1.

Define the totalized product

$$\pi_1 \odot \pi_2 = \begin{cases} \mathsf{compact}(\pi_1 \cdot \pi_2), & \text{if raw composition succeeds,} \\ \mathbf{0}, & \text{otherwise,} \end{cases}$$

$$\mathbf{0} \odot \pi = \pi \odot \mathbf{0} = \mathbf{0}.$$

This is the operation used while reducing a source sequence.

Worked derivation. With

```
literal integer ( -- n )
+               ( n n -- n )
DUP             ( x[1] -- x[1] x[1] )
```

the fragment `1 2 + DUP` has the derivation

$$(- - n) \cdot (- - n) = (- - n\ n),$$

$$(- - n\ n) \cdot (n\ n - - n) = (- - n),$$

$$(- - n) \cdot (x[1] - - x[1]\ x[1]) = (- - n[1]\ n[1]).$$

The last step refines `x` to `n` and propagates that refinement to both outputs of `DUP`. Running this example through the native evaluator with `ex1types.txt` and `ex1specs.txt` confirms the calculation:

```
1    ( -- n )
2    ( -- n )
+    ( n n -- n[1] )
DUP  ( n[1] -- n[1] n[1] )
< n[1] n[1]
```

Proposition 1 (Determinacy of raw composition) *For fixed freshening and fresh-index policies, raw boundary composition either returns one effect or clashes. Different choices of fresh positive indices produce alpha-equivalent effects.*

Proof. At each recursive step there is one pair of top boundary symbols. The comparable meet, when it exists, is unique. Substitution and removal therefore determine the next state. The recursion decreases $|R_1| + |L_2|$ by two and terminates. Fresh-index choices can change only variable names, hence the alpha-equivalence result. □

This proposition applies to raw effects. Algebraic associativity of an ideal stack-effect product motivates the polycyclic-monoid account in [5, 7]; however, inserting a lossy compaction after every binary product can affect later identity information. Consequently, algebraic laws of the raw domain do not automatically hold for the stored representation.

5 Normalization and Loss of Correlation

The Gforth implementation mutates cloned working effects during freshening and substitution. After a successful operation it normalizes the result in two passes:

1. count occurrences of every symbolic identity over the result and the associated working sequence;
2. traverse in deterministic stack order, assign small consecutive indices, and substitute the normalized symbols.

An index remains visible if it was explicit in source or occurs more than twice. An implicitly generated identity occurring at most twice is printed without an index. Thus

$$(n[7] - -n[7]) \quad \longmapsto \quad (n - -n)$$

when index 7 was implicit, while an explicit index in a specification is retained.

Remark 1 (Compaction is a widening) *Removing an index preserves stack shape and per-position type constraints but forgets an equality constraint. Consequently* compact *is deterministic and concise, but neither injective nor semantics-preserving for symbolic identity. The printed effect constitutes a conservative weakening of the raw effect with respect to correlation information.*

This design choice affects the analysis because compacted effects are stored and reused rather than being confined to output formatting. A formalization of the artifact must therefore define operators as "raw operation followed by compaction," rather than proving theorems only about an uncompacted idealization.

6 Branch Contracts

The branch operator combines effects that can be given one common contract. It is called `glb` in the code and written $\sqcap$ here. Its executable definition is more precise than the phrase "pointwise meet."

Let $\pi = (L - -R)$ and $\theta = (M - -N)$. If $|L| > |M|$, designate π the long effect and set $k = |L| - |M|$. Compatibility first requires

$$|R| - |N| = k.$$

The first k symbols of the long input and output are then unified pairwise. They represent a frame prefix preserved by the long effect. After that prefix is constrained, the remaining input slices and output slices of the two effects must have equal lengths and are unified pointwise. The symmetric rule applies when $|M| > |L|$. Any incomparable type causes failure.

For example, the effects

$$(x\ a\text{-}addr - -\epsilon), \qquad (x\ c\text{-}addr - -\epsilon)$$

combine to

$$(x\ a\text{-}addr - -\epsilon)$$

because `a-addr` is the stronger requirement and is accepted by both store operations represented by the branches. With an `IF` effect $(flag - -\epsilon)$, the whole conditional contract additionally consumes the flag according to its declarative effect template.

The order-theoretic terminology requires care. If an effect is viewed as a *contract*, a lower element can mean a stronger precondition and stronger symbolic facts. A branch merge seeks a contract valid for both alternatives; it differs from the union of concrete transition relations. The present implementation defines this partial contract operator syntactically by prefix preservation and unification. A full semantic soundness theorem would require an explicit operational semantics for every declared primitive and a variance-aware contract order; that theorem is outside the current artifact.

7 One-Step Approximation of Iteration

A general abstract interpreter computes a post-fixpoint at a loop header, usually with widening when the abstract domain has infinite ascending chains [1]. This checker instead uses a local operation inherited from the stack-effect calculus:

Definition 3 (One-step repeated summary) *For an effect π, define*

$$\pi_2^\star = \pi \sqcap (\pi \cdot \pi)$$

when both raw composition and the effect meet succeed; compact the result before storing it.

The operation compares one execution with two executions. It is a finite, local test that rejects bodies whose stack shape changes at this step. It is strictly weaker than computing

$$\bigwedge_{j \geq 0} \pi^j$$

or proving an invariant I satisfying an appropriate inductive condition. In particular, existence of a common lower bound for π and π^2 alone does not establish stability for every π^j. The notation $\pi_2^\star$ is used here to avoid confusing the implementation with Kleene star.

The native word `ev-spec-pistar` implements the one-versus-two rule above, and declarative `repeat` evaluates through that path. It does not invoke a separate general invariant or idempotent-closure algorithm. Claims that all accepted loops have a mathematically idempotent summary therefore need stronger hypotheses than the current checker enforces.

8 Declarative Source Semantics

The effect algebra is only one layer of a source checker. Forth parser words can consume names or delimited text, definitions extend the dictionary, and control words are immediate. The specification format attaches metadata such as

```
"("      parse until ")"       ( -- )
CONSTANT parse word define     ( x -- )
VARIABLE parse word define     ( -- a-addr )
:        parse word define colon ( -- )
IF       control if            ( flag -- )
```

Ordinary consumed comments are not trusted as type declarations. Thus a source comment `( - n )` cannot override inference. Recognized locals syntax may provide a provisional definition shape.

Structured controls are supplied through paired `syntax:` and `effect:` descriptions. A conditional can be specified as

```
syntax:
  IF <then branch> [ELSE <else branch>] FI
effect:
  IF
  either <then branch> <else branch>
```

The effect-template language is

$$E ::= \epsilon \mid \langle s \rangle \mid c \mid E_1 E_2 \mid \mathsf{either}(E_1, E_2) \mid \mathsf{repeat}(E),$$

where $\langle s \rangle$ refers to a captured source segment and c is a control-word runtime effect. Its implemented semantics is

$$\begin{aligned} [\![\epsilon]\!] &= \mathbf{1}, \\ [\![\langle s \rangle]\!] &= X_s, \\ [\![c]\!] &= \Gamma(c), \\ [\![E_1 E_2]\!] &= [\![E_1]\!] \odot [\![E_2]\!], \\ [\![\mathsf{either}(E_1, E_2)]\!] &= \overline{\sqcap}([\![E_1]\!], [\![E_2]\!]), \\ [\![\mathsf{repeat}(E)]\!] &= \overline{\star}_2([\![E]\!]), \end{aligned}$$

where bars denote totalization with $\mathbf{0}$ and include compaction after a successful raw operation.

9 The `ex1` Input/Output Example

The bundled `real` profile gives a complete, reproducible example of the evaluator interface. It consists of the three files `ex1types.txt`, `ex1specs.txt`, and `ex1prog.txt`. The launcher command is

```
./run-evaluator-gforth.sh real
```

or, equivalently, an explicit invocation of `gforth-evaluator.fs` with `-types`, `-specs`, and `-prog` arguments.

9.1 Type input

The type file declares canonical names, aliases, subtype edges, and scanner delimiters. Its essential declarations are:

```
type X cell x
type n number N
type char c
type flag boolean f
type addr a
type c-addr ca
type a-addr aa
type xt

rel n < X
rel flag < n
rel addr < X
rel char < n
rel c-addr < addr
rel a-addr < c-addr
rel xt < X
scanner EOL "\n"
```

For example, `X`, `cell`, and `x` name one canonical type, while `a-addr` is more precise than `c-addr`. The loader computes the transitive closure, so it also knows $a\text{-}addr < addr < X$.

9.2 Specification input

The specification file supplies both runtime effects and source-processing metadata. Representative entries are:

```
DUP      ( X[1] -- X[1] X[1] )
SWAP     ( X[2] X[1] -- X[1] X[2] )
"("      parse until ")" ( -- )
:        parse word define colon ( -- )
;        control end ( -- )
literal integer ( -- n )
IF       control if ( flag -- )
ELSE     control else ( -- )
FI       control fi ( -- )
BEGIN    control begin ( -- )
WHILE    control while ( flag -- )
REPEAT   control repeat ( -- )
UNTIL    control until ( flag -- )
@        ( a-addr -- X )
C@       ( c-addr -- char )
0=       ( n -- flag )
```

Thus the same input describes ordinary words, literal recognition, parser words, defining words, and control roles. The indexed effects of `DUP` and `SWAP` preserve correlations; `@` and `C@` exercise the address subtype chain.

9.3 Program input and inferred output

The program file defines five words and then checks one top-level sequence:

```
: PEEK2 DUP @ SWAP @ ;
: MAYSWAP IF SWAP FI ;
: NLOOP BEGIN DUP 0= WHILE REPEAT ;
: ULOOP BEGIN DUP 0= UNTIL ;
: KEEPIF IF DUP ELSE DUP FI ;

DP DP C@ 0= KEEPIF
```

The source layout is immaterial outside parsed strings and comments. The native run writes inferred definitions to `ex1prog.txt.log`:

```
PEEK2   ( a-addr[1] -- X[2] X )
MAYSWAP ( X[1] X[1] flag -- X[1] X[1] )
NLOOP   ( n[1] -- n[1] )
ULOOP   ( n[1] -- n[1] )
KEEPIF  ( X[1] flag -- X[1] X[1] )
```

The top-level trace ends with

```
DP      ( -- a-addr[1] )
DP      ( -- a-addr )
C@      ( a-addr -- char )
0=      ( char -- flag )
KEEPIF  ( a-addr[1] flag -- a-addr[1] a-addr[1] )
< a-addr[1] a-addr[1]
```

Type refinement occurs at `C@`: the producer yields `a-addr`, while `C@` requires `c-addr`. Since *a-addr* $<$ *c-addr*, composition succeeds and retains the more precise address information. The flag from `0=` is then consumed by `KEEPIF`; both branches duplicate the surviving address, yielding the final pair of correlated `a-addr` cells. The current native evaluator was run on both this complete profile and the command-line example in Section 4; both produced the outputs shown above.

9.4 Acyclic effect equations

Fix a finite parsed tree T for one source body and environment Γ. Assume every undecomposed leaf has already been assigned an effect by word lookup, literal classification, or an earlier definition. Associate a variable X_A with every tree node A:

$$\begin{aligned} X_A &= \Gamma(A) && \text{if } A \text{ is a leaf,} \\ X_A &= X_B \odot X_C && \text{if } A \text{ sequences children } B, C, \\ X_A &= \widehat{[\![E]\!]}_X && \text{if } A \text{ applies control template } E. \end{aligned}$$

Every right-hand side mentions only proper descendants, so the dependency graph is acyclic.

Theorem 1 (Exact characterization of parsed-body checking) *Let T and Γ satisfy the assumptions above. The induced equation system has exactly one solution in*

$$D = \{\textit{compacted effects}\} \cup \{\mathbf{0}\}.$$

For every segment A, the body evaluator returns its solution value X_A; it reports a clash at A exactly when $X_A = \mathbf{0}$. The body is accepted if and only if the root value is nonzero.

Proof. Proceed by induction on node height. A leaf is fixed by Γ. At a sequence node, the induction hypothesis uniquely fixes both child values and

$\odot$ is a deterministic total function on D. At a control node, structural induction on E uniquely determines the result because each constructor invokes a deterministic totalized operator. The evaluator visits the same children and applies the same operator. Therefore it computes the unique equation value at every node, including the root. $\square$

This is an exactness theorem for the implemented, post-parse computation; it is not a conventional progress-and-preservation theorem. It does not cover the outer interpreter's evolving dictionary, provisional forward-reference summaries, recursion, or the semantic correctness of primitive specifications.

10 Native Gforth Implementation

The implementation is the single source file `gforth-evaluator.fs`. Its word families mirror the formal layers:

Formal role	Representative Gforth words
canonical types and relation	`ev-ts-*`
typed indexed symbols	`ev-sym-*`
stack-effect records	`ev-spec-*`
linear reduction and normalization	`ev-spec-list-*`
specification dictionary	`ev-ss-*`
source scanning and program parsing	`ev-sc-*`, `ev-parse-*`
diagnostics and log output	`ev-diag-*`, `ev-log-*`
command-line driver	`ev-parse-args`, `ev-run-native`

Strings, growable vectors, source spans, type symbols, effects, and control expressions are represented by explicitly allocated Gforth records. The checker normalizes CR, LF, and CRLF input, clones mutable dictionary effects before evaluation, allocates disjoint wildcard ranges, and builds the final annotation from the same normalized working value as the final effect. The launcher reserves a 4 MiB dictionary.

For effects of visible sizes m and n, boundary matching itself performs at most $\min(m, n)$ recursive matches. The actual cost is higher because each match substitutes through the current working vectors. With this representation, reducing a program of N words is roughly quadratic in the largest intermediate symbolic effect in the worst case. A union–find representation for identities and persistent stack rows would reduce copying.

Mutation imposes three implementation obligations:

1. dictionary effects must be deep-cloned before evaluation;
2. independently evaluated branches need disjoint wildcard ranges;
3. normalization and wildcard allocation must be deterministic across repeated native runs.

All three are handled explicitly by the current Gforth source.

11 Artifact Evaluation

The checked-in corpus has 29 programs: 13 positive and 16 negative. The positive set consists of seven Forth-2012-profile examples, five custom-profile examples, and one article profile. A fresh batch execution of the current `gforth-evaluator.fs` accepts all 13 positive programs. Fourteen of the 16 negative programs terminate with diagnostics. The two misses are `linear_clash.fs` and `top_level_clash.fs`, both involving the linear fragment `DP 1+`. These cases identify gaps in top-level linear checking.

Corpus class	Programs	Expected	Current Gforth result
Forth-2012 positive	7	accept	7 accepted
Custom-profile positive	5	accept	5 accepted
Article positive	1	accept	1 accepted
Negative	16	reject	14 rejected, 2 missed

The declarative Forth-2012 environment contains 50 lines of type declarations and 278 lines of vocabulary/control specifications. The current native checker is 4,424 lines of Gforth. These counts establish nontrivial scale, but they are not a performance benchmark or a soundness evaluation. In particular, the corpus is regression evidence for the implemented semantics; it cannot expose an error shared by the checker and its specifications.

12 Relation to Other Typed Stack Systems

Forth and algebraic stack effects. Forth standard practice uses stack comments extensively, but the standard does not turn every such comment into a static typing judgment [3, 11]. Earlier stack-effect work introduced algebraic composition, multiple effects, typed wildcards, and must-analysis operators [5, 6, 8, 9]. The present artifact combines those mechanisms with source parsing, external control declarations, inferred user definitions, and diagnostics.

Compositional low-level typing. Saabas and Uustalu give compositional type systems for stack-based low-level languages [12]. Their setting is closer to fixed low-level instruction syntax. This project instead treats parser behavior, defining words, and control syntax as profile data, as required for source-level Forth analysis.

WebAssembly 3.0. The latest online WebAssembly Core Specification available while preparing this version identifies itself as Release 3.0 dated 23 July 2026 [14]. WebAssembly validation uses typed instruction sequences, structured control signatures, and stack-polymorphic treatment of unreachable code. Its language and control forms are fixed by the specification and have substantial mechanized metatheory [15, 4]. By contrast, this checker loads a nominal type relation and source grammar from files and tracks symbolic cell identity. The external specification format supports retargeting, but the current loop rule and metatheory provide fewer guarantees than WebAssembly validation.

First-class concatenative functions. Stack-polymorphic higher-order words require variables ranging over stack rows rather than over individual cells. Stoddart and Knaggs studied type inference for stack-based languages [13]. A recent 2025 account of the IV language makes the modern design trade-off explicit: it avoids rank-n inference for first-class functions by requiring definition annotations, extending operator types across stack depths, and using explicit parameters for execution and composition operators [2]. The checker in this project has indexed cell polymorphism and an implicit untouched frame, but no first-class quotation type or general row-variable unification. Adding quotations would therefore require an extension of the type language rather than additional vocabulary entries alone.

13 Limitations and Research Directions

The implementation realizes a restricted static analysis with the following scope limitations.

- Acceptance is relative to external primitive specifications. An incorrect effect declaration can make the analysis unsound with respect to concrete execution.
- Comparable meet rejects incomparable types even if the declared poset has a meaningful lower bound represented by a third type.

- Compaction loses pairwise implicit correlations and may reduce the precision of later compositions.
- Branch meet supplies a syntactic common contract; a full semantic theorem needs variance-aware contracts and an operational semantics.
- $\pi_2^\star$ is not a general loop invariant computation.
- Non-local effects such as `THROW`, return-stack manipulation, dynamic execution, and unrestricted metaprogramming are approximated or outside the model.
- The parsed-body theorem excludes recursive dictionary construction and forward-reference machinery.
- Two checked-in top-level negative tests are not currently rejected.

Potential technical extensions include preserving raw correlations internally and compact only for presentation; represent stack tails with explicit row variables; replace one-step repetition with control-flow fixpoint computation; model effect contracts with explicit pre/post variance; and mechanize the raw calculus and parser-to-equation translation. A union–find implementation of symbolic identities would also avoid repeated whole-sequence substitution.

14 Conclusion

Static checking of a concatenative language can be organized around symbolic stack transformations. In this project, subtype-aware boundary unification handles sequencing, prefix-aware effect meet handles branches, and a local one-versus-two comparison approximates repetition. External specifications make the same engine applicable to multiple Forth-like profiles and custom control syntaxes.

These three operations do not constitute a complete type system for arbitrary Forth. They define a compact, deterministic checker whose implemented behavior can be characterized exactly on parsed bodies. The characterization distinguishes raw effects, compaction, and loop approximation. Typed symbolic stack effects can therefore serve as a basis for extensions involving row polymorphism, semantic soundness, and fixpoint-based control analysis.

References

[1] P. Cousot and R. Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *POPL*, pp. 238–252, 1977. doi:10.1145/512950.512973.

[2] A. M. Dinikeev. Type system for a statically typed concatenative programming language with first class function support. *Information-measuring and Control Systems*, no. 5, pp. 15–25, 2025. doi:10.18127/j20700814-202505-02.

[3] Forth 200x Standardisation Committee. *Forth 2012 Standard*. Draft proposed standard, 10 November 2014. `https://forth-standard.org/standard/intro`.

[4] D. McDermott, Y. Morita, and T. Uustalu. A type system with subtyping for WebAssembly's stack polymorphism. In *ICTAC 2022*, LNCS 13572, pp. 305–323, 2022. doi:10.1007/978-3-031-17715-6_20.

[5] J. Pöial. Algebraic specifications of stack effects for Forth programs. In *1990 FORML Conference Proceedings, EuroFORML'90*, pp. 282–290, 1991.

[6] J. Pöial. Multiple stack-effects of Forth programs. In *1991 FORML Conference Proceedings, euroFORML'91*, pp. 400–406, 1992.

[7] J. Pöial. Algebraic specifications of stack effects. *Forth Dimensions*, 16(4):18–20, November/December 1994. `https://www.forth.org/fd/FD-V16N4.pdf`.

[8] J. Pöial. Stack effect calculus with typed wildcards, polymorphism and inheritance. Abstract in *18th EuroForth Conference*, p. 38, 2002; accompanying presentation slides. `https://www.complang.tuwien.ac.at/anton/euroforth/ef02/poial02.pdf`.

[9] J. Pöial. Typing tools for typeless stack languages. In *Proceedings of the 22nd EuroForth Conference*, Cambridge, England, pp. 40–46, 2006. `https://www.complang.tuwien.ac.at/anton/euroforth/ef06/poial06.pdf`.

[10] J. Pöial. Java framework for static analysis of Forth programs. In *Proceedings of the 24th EuroForth Conference*, Vienna, Austria, pp. 20–23, 2008. `https://www.complang.tuwien.ac.at/anton/euroforth/ef08/papers/poial.pdf`.

[11] E. D. Rather, D. R. Colburn, and C. H. Moore. The evolution of Forth. In *History of Programming Languages—II*, pp. 625–670, ACM, 1996. doi:10.1145/234286.1057832.

[12] A. Saabas and T. Uustalu. Compositional type systems for stack-based low-level languages. In J. Gudmundsson and C. B. Jay, editors, *Proceedings of CATS 2006*, CRPIT 51, pp. 27–39. Australian Computer Society, 2006. `https://cs.ioc.ee/~tarmo/papers/saabas-uustalu-cats06.pdf`.

[13] B. Stoddart and P. J. Knaggs. Type inference in stack based languages. *Formal Aspects of Computing*, 5(4):289–298, 1993. doi:10.1007/BF01212404.

[14] WebAssembly Community Group. *WebAssembly Core Specification, Release 3.0.* Editor: A. Rossberg, 23 July 2026. `https://webassembly.github.io/spec/core/`.

[15] C. Watt. Mechanising and verifying the WebAssembly specification. In *CPP 2018*, pp. 53–65, ACM, 2018. doi:10.1145/3167082.